

Computer Graphics

- Model Loading

CSE352 – Spring'2026

Slides adapted from CG course by Prof. Kai Lawonn at FSU Jena
https://vis.uni-jena.de/?page_id=540

Load Models from Files

- So far, geometry has been defined directly in code — hardcoded arrays of vertices and indices:

```
const vertices = [  
    0.0,  0.5, 0.0,    // top  
    -0.5, -0.5, 0.0,  // bottom-left  
    0.5,  -0.5, 0.0   // bottom-right  
];
```

- This works for simple primitives (triangles, cubes, spheres) but breaks down fast — a realistic character mesh has hundreds of thousands of vertices
- Real assets are created by artists in dedicated tools: Blender, Maya, ZBrush → the renderer's job is to load and display them, not to define them

Load Models from Files

- Separating content from code also means:
 - Artists iterate on models without touching the renderer
 - The same renderer loads any model — not just hardcoded ones
 - Models can be swapped, updated, or streamed at runtime
- A file format is the contract between the authoring tool and the renderer
- OBJ is the simplest widely-supported format — a natural first step before more complex formats like glTF or FBX

What is OBJ?

- OBJ is a plain-text 3D geometry format developed by Wavefront Technologies in the 1980s
- Now a universal interchange format — exported by Blender, Maya, 3ds Max, etc.
- Stores geometry only: positions, normals, UVs, faces
- Material references live in a separate `.mtl` file
- Human-readable: open any `.obj` in a text editor and it makes sense

OBJ: Positions, Normals, UVs

- Each geometric attribute is declared on its own line with a keyword prefix

```
v  0.0  1.0  0.0      # vertex position  (x y z)
vn 0.0  1.0  0.0      # vertex normal    (nx ny nz)
vt 0.5  1.0           # texture coord    (u v)
```

- These are global lists — 1-indexed, order matters
- Vertex colors can be appended to v lines (non-standard but common):

```
v  0.0  1.0  0.0  1.0  0.0  0.0  # position + RGB color
```

- Normals are not required — some exporters omit them (flat shading or auto-computed)
- UVs are not required — models without textures often omit them

OBJ: Faces

- The f keyword defines a face using indices into the v / vt / vn lists

```
f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3
```

- Indices are 1-based (not 0-based like arrays)
- Slashes separate position / texcoord / normal index; parts can be omitted:

```
f 1 2 3 # positions only  
f 1//1 2//2 3//3 # positions + normals, no UVs  
f 1/1/1 2/2/2 3/3/3 # all three
```

- Faces can be quads or n-gons — loaders fan-triangulate them:

```
f 1/1/1 2/2/2 3/3/3 4/4/4 # quad → two triangles
```

- Negative indices are relative (−1 = last declared):

```
f -3/-3/-3 -2/-2/-2 -1/-1/-1
```

OBJ: Material References & Groups

- `mtllib` links to one or more `.mtl` files:

```
mtllib scene.mtl
```

- `usemtl` activates a material for all subsequent faces:

```
usemtl Body
f 1/1/1 2/2/2 3/3/3
f 4/4/4 5/5/5 6/6/6

usemtl Glass
f 7/7/7 8/8/8 9/9/9
```

- `g` (group) and `o` (object) name sub-meshes — often ignored by simple loaders
- A single OBJ file can contain multiple objects and materials
- The order of `usemtl` calls determines which index ranges belong to each material

OBJ: Minimal Complete Example

```
# A single triangle with UVs and normals
mtllib mesh.mtl

v -1.0 0.0 0.0
v 1.0 0.0 0.0
v 0.0 1.0 0.0

vn 0.0 0.0 1.0

vt 0.0 0.0
vt 1.0 0.0
vt 0.5 1.0

usemtl Red
f 1/1/1 2/2/1 3/3/1
```

- All three vertices share the same normal (index 1)
- Each vertex has its own UV

MTL: Structure & Standard Properties

- MTL (*Material Template Library*) is a companion plain-text format
- Each material block begins with `newmtl`:

```
newmtl Body
  Ka  0.2  0.2  0.2      # ambient  color  (r g b)
  Kd  0.8  0.4  0.1      # diffuse  color  (r g b)
  Ks  1.0  1.0  1.0      # specular color  (r g b)
  Ke  0.0  0.0  0.0      # emissive color  (r g b)
  Ns  64.0                # specular exponent (shininess)
  Ni  1.45                # index of refraction
  d   1.0                # opacity  (1 = fully opaque)
  illum 2                # illumination model (2 = Phong)
```

- `illum` values: 0 = constant color, 1 = Lambertian, 2 = Phong + specular
- Multiple `newmtl` blocks can appear in one `.mtl` file

MTL: Texture Maps

- Texture paths are appended to map_ keywords:

```
map_Kd    textures/body_albedo.png    # diffuse / albedo
map_Ks    textures/body_specular.png  # specular
map_Ke    textures/body_emission.png  # emissive
map_d      textures/body_opacity.png  # alpha / opacity
map_Bump   textures/body_normal.png    # normal map (also: norm, bump)
disp      textures/body_height.png     # displacement / height
```

- PBR extensions (non-standard but widely supported):

```
map_Pm    textures/body_metallic.png  # metallic
map_Pr     textures/body_roughness.png # roughness
map_Ka     textures/body_ao.png        # ambient occlusion
```

- Any map_ entry is optional — missing = use scalar value instead
- Paths are relative to the .mtl file location

Loader: Parsing Strategy

- The loader reads line-by-line, dispatching on the first keyword token
- Four local arrays accumulate raw OBJ data (1-indexed, sentinel at [0]):

```
const positions = [[0,0,0]]; // v
const texcoords = [[0,0]]; // vt
const normals = [[0,0,1]]; // vn
const colors = [[1,1,1]]; // per-vertex color from v lines
```

- On each f line, face vertices are fan-triangulated and emitted as triples:

```
// quad [A, B, C, D] → triangles [A,B,C] and [A,C,D]
for (let i = 1; i < parts.length - 1; i++) {
  addVertex(parts[0]);
  addVertex(parts[i]);
  addVertex(parts[i + 1]);
}
```

- Negative indices are resolved before lookup:

```
function resolveIndex(i, array) {
  const idx = parseInt(i);
  return idx >= 0 ? idx
    : array.length + idx;
}
```

Loader: Vertex Deduplication

- OBJ can share positions but give each face-corner unique normals/UVs
- A GPU vertex buffer needs one entry per unique (position, uv, normal) combination
- The loader deduplicates using a Map keyed on the full face-corner string:

```
const addVertex = (vStr) => {  
  const key = vStr + '|' + currentMaterial; // e.g. "12/5/3|Body"  
  if (vertexMap.has(key)) {  
    indices.push(vertexMap.get(key)); // reuse existing vertex  
    return;  
  }  
  const index = indexCounter++;  
  vertexMap.set(key, index);  
  indices.push(index);  
  // ... unpack and push position, uv, normal into flat arrays  
};
```

- The material name is included in the key because the same geometric vertex used under two materials must become two separate GPU vertices

Loader: Material Ranges

- To draw each material as a separate draw call we record which index ranges belong to which material while parsing faces:

```
const addFace = (parts) => {
  const start = indices.length;
  for (let i = 1; i < parts.length - 1; i++) {
    addVertex(parts[0]); addVertex(parts[i]); addVertex(parts[i + 1]);
  }
  if (curMaterial) {
    if (!materialRanges[curMaterial]) materialRanges[curMaterial] = [];
    materialRanges[curMaterial].push({
      start,
      count: indices.length - start
    });
  }
};
```

- Result: `geometry.materialRanges["Body"] = [ {start:0, count:300}, ... ]`
- At render time: `bind material → drawElements(start, count)` for each range

Using the Loader

- - The JS loader is async because fetching .mtl files may involve network I/O:

```
// Default fetch callback uses the browser's fetch()
const objText = await fetch('mesh.obj').then(r => r.text());
const model   = await Model.load(objText);

// Custom fetch (e.g. path remapping, caching)
const model = await Model.load(objText, (name) =>
    fetch('/assets/models/' + name).then(r => r.text()) );
```

- After loading, geometry is GPU-ready:

```
gl.bufferData(gl.ARRAY_BUFFER,
    new Float32Array(model.geometry.position), gl.STATIC_DRAW);

gl.bufferData(gl.ELEMENT_ARRAY_BUFFER,
    new Uint32Array(model.geometry.indices), gl.STATIC_DRAW);
```

Using the Loader: Accessing Materials

- Materials are stored in a plain object keyed by name:

```
const mat = model.materials['Body'];

// Scalar / color properties are always present
console.log(mat.Kd); console.log(mat.Ns);
// Texture paths are null if not specified in the MTL
if (mat.albedo)    loadTexture(mat.albedo);
...
```

- Per-material index ranges for multi-material draw calls:

```
for (const [name, ranges] of Object.entries(model.geometry.materialRanges)) {
  const mat = model.materials[name];
  bindMaterial(mat);                // set uniforms / textures

  for (const { start, count } of ranges)
    gl.drawElements(gl.TRIANGLES, count, gl.UNSIGNED_INT, start * 4);
}
```

Geometry Layout Summary

- All per-vertex arrays are flat, parallel, and indexed by `geometry.indices`

Attribute	Array	Stride	Source
position	<code>geometry.position</code>	3 floats	v lines
texcoord	<code>geometry.texcoord</code>	2 floats	vt lines (0,0 if absent)
normal	<code>geometry.normal</code>	3 floats	vn lines (0,0,1 if absent)
color	<code>geometry.color</code>	3 floats	v r g b (1,1,1 if absent)

- Vertex `i` occupies:

```
position[i*3 .. i*3+2]
texcoord[i*2 .. i*2+1]
normal   [i*3 .. i*3+2]
tangent  [i*3 .. i*3+2]
```

- Interleaving is not done by the loader — do it manually if your pipeline requires it

Limitations & Common Pitfalls

- N-gon faces are fan-triangulated — concave polygons will tessellate incorrectly
→ triangulate your mesh before export (Blender: Mesh → Faces → Triangulate)
- OBJ has no skinning, animation, or scene hierarchy — use glTF for those
- Texture paths in MTL are relative to the MTL file, not the OBJ file → use the fetch callback to remap paths as needed
- The same vertex used under two materials produces two GPU vertices (by design — needed for correct draw call splitting)
- Some exporters write Windows-style backslash paths in MTL — sanitize if needed: `const path = mat.albedo.replace(/\\/g, '/');`

Sample Program

Loading & Viewing an OBJ Model

Program Overview

- The program loads a real-world OBJ model and renders it with Phong shading and diffuse + specular texture maps
- Mouse drag rotates the model via arcball rotation — the standard technique used in 3D viewers (Blender viewport, MeshLab, etc.)
- Architecture: one class (MainApp) owns everything — WebGL context, shader, VAO, textures, matrices, and input state

Initialization

Initialization sequence:

1. Get WebGL2 context
2. Fetch and compile shaders
3. Fetch OBJ → parse with Model loader → upload geometry to GPU (VAO)
4. Load textures
5. Set constant uniforms (model matrix, projection, light)
6. Enter render loop

```
initWebGL()           ← async, awaits shaders + VAO
└─ initVAO()          ← fetches OBJ/MTL, builds VAO
└─ initTextures()     ← creates Texture objects
└─ initUniforms()     ← sets one-time uniforms
```

Loading the Model (initVAO)

- OBJ and MTL are fetched from a local model/ folder using the Model loader:

```
const objResponse = await fetch(this.modelPath + 'survival_guitar_backpack.obj');
const objText = await objResponse.text();

const model = await Model.load(objText, (name) =>
    fetch(this.modelPath + name).then(r => r.text())
);

this.mesh      = model.geometry; // arrays: position, texcoord, normal, indices
this.materials = model.materials; // map of material name → Material
```

- Bounding box is used for auto-fit (centering + scaling the model matrix):

```
this.size    = [max.x - min.x, max.y - min.y, max.z - min.z];
this.center  = [(max.x+min.x)/2, (max.y+min.y)/2, (max.z+min.z)/2];
```

Loading the Model (initVAO)

- Geometry is uploaded into three separate VBOs + one index buffer, all bound into a single VAO:

Attribute	VBO	Size
aPos	positionVBO	3 floats
aTexCoord	texcoordVBO	2 floats
aNormal	normalVBO	3 floats
	indexVBO	Uint16

- Note: Uint16 Array for the index buffer → max 65535 unique vertices. Use Uint32Array + `gl.UNSIGNED_INT` for larger models.

Model & Projection Matrices (initUniforms)

- The model is auto-fitted to a 2-unit cube regardless of its original scale:

Step 1 — translate: moves the model's center to the origin

Step 2 — scale: fits the longest axis into [-1, +1]

```
const scale = 2.0 / Math.max(size.x, size.y, size.z);  
mat4.fromScaling(model, [scale, scale, scale]);  
mat4.translate(model, model, [-center.x, -center.y, -center.z]);
```

- Projection is standard perspective, set once:

```
mat4.perspective(projection,  
    toRadian(45),          // 45° vertical FOV  
    canvas.width / canvas.height,  
    0.1, 100.0  
);
```

- These never change during the render loop — set as uniforms once in `initUniforms()`
- The view matrix and light position update every frame (arcball rotation)

Textures & Materials

- The loader returns all materials; the program uses the first one:

```
const mat = Object.values(this.materials)[0];
```

- Diffuse and specular map filenames come from the parsed MTL:

mat.albedo → e.g. "backpack_diffuse.jpg" (map_Kd)

mat.specular → e.g. "backpack_specular.jpg" (map_Ks)

- Texture objects are constructed with a fallback placeholder color in case the file is missing or still loading:

[illegible]

Textures & Materials

- Sampler uniforms are bound by name to match the shader's material struct:

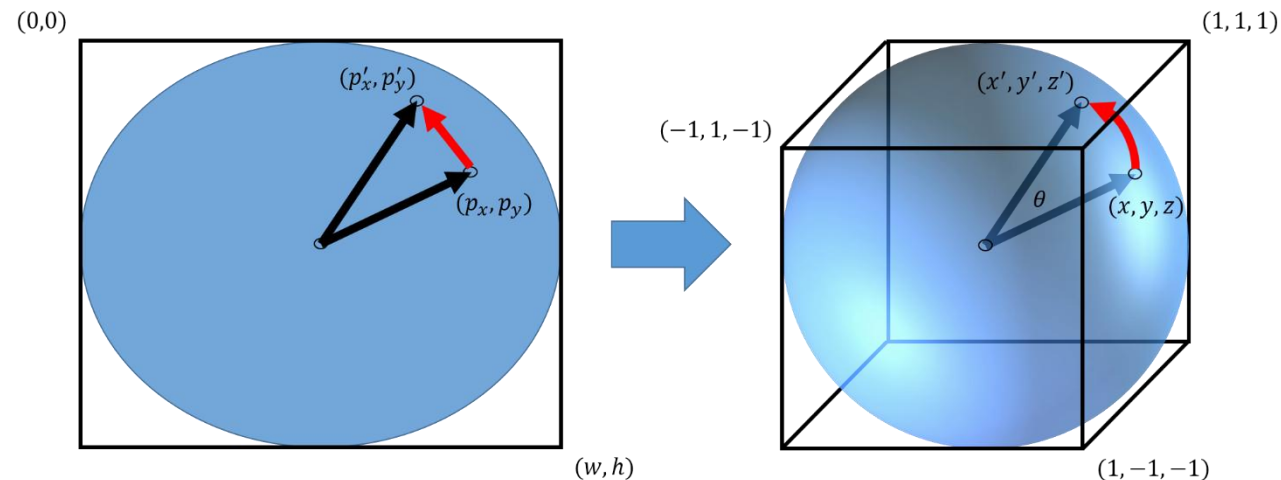
```
diffuseMap.setSamplerUniform(shader, 'material.diffuse');    // → sampler2D  
specularMap.setSamplerUniform(shader, 'material.specular');  // → sampler2D
```

- Light and shininess uniforms complete the Phong setup:

```
shader.setFloat('material.shininess', 32.0);  
shader.setVec3v('light.ambient',  [0.2, 0.2, 0.2]);  
shader.setVec3v('light.diffuse',   [0.8, 0.8, 0.8]);  
shader.setVec3v('light.specular',  [1.0, 1.0, 1.0]);
```

Arcball Rotation: Concept

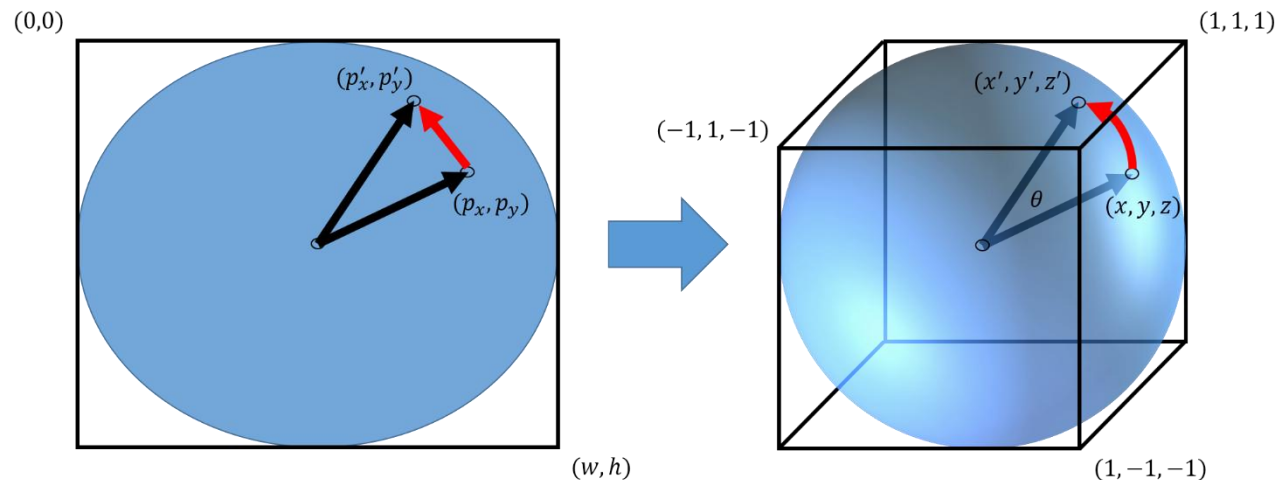
- **Problem:** map a 2D mouse drag to a meaningful 3D rotation
- Naive Euler angles (yaw/pitch from dx/dy) suffer from gimbal lock and accumulate floating-point drift — not suitable for interactive viewers



- **Arcball solution:** imagine a hemisphere sitting over the viewport. Mouse clicks/drags map to points on this hemisphere. The rotation is derived from the arc between two surface points.

Arcball Rotation: Concept

- The axis of rotation is the cross product of the two surface vectors
- The angle is derived from their dot product (= cosine of angle between them)
- The result is a **quaternion** — composable, drift-free, gimbal-lock-free



- If the mouse is inside the unit circle $\rightarrow$ project onto the sphere surface
- If outside $\rightarrow$ project onto the equator edge (prevents wild rotations)

Arcball Rotation: Screen-to-Sphere Mapping

- Mouse pixel coordinates are first normalized to $[-1, +1]$ NDC:

```
screenToArcball(x, y) {  
  const nx = (2 * x) / canvas.clientWidth - 1;  
  const ny = -(2 * y) / canvas.clientHeight + 1; // Y flipped  
  const len2 = nx*nx + ny*ny;  
  
  if (len2 <= 1.0) { // inside unit circle → on the sphere hemisphere  
    return [nx, ny, Math.sqrt(1.0 - len2)];  
  } else { // outside unit circle → clamp to equator  
    const n = Math.sqrt(len2);  
    return [nx/n, ny/n, 0];  
  }  
}
```

- - When $\text{len}^2 \leq 1$: $z = \sqrt{(1 - x^2 - y^2)}$ → the point lies on the unit sphere
- - When $\text{len}^2 > 1$: normalize (x, y) to the edge → $z = 0$ (equator fallback)
- - Result is a 3D unit vector pointing from the sphere center to the surface

Arcball Rotation: Quaternion Composition

- On mousedown: record the start position and snapshot the current rotation

```
arcball.startPos      = screenToArcball(e.offsetX, e.offsetY);  
arcball.baseRotation = copy(arcball.rotation);    // snapshot
```

- On mousemove: compute the delta quaternion from start → current position

```
const currPos = screenToArcball(e.offsetX, e.offsetY);  
  
const axis = cross(startPos, currPos);           // rotation axis  
const dot  = clamp(dot(startPos, currPos), -1, 1);  
const angle = acos(dot) * sensitivity;           // rotation angle  
  
if (length(axis) > 1e-6) {  
    const delta = quatFromAxisAngle(axis, angle);  
    // compose delta on top of the base snapshot – not the live rotation  
    arcball.rotation = multiply(delta, arcball.baseRotation);  
}
```

Arcball Rotation: Quaternion Composition

- Key insight: always composing $\text{delta} \times \text{baseRotation}$ (the mousedown snapshot), NOT $\text{delta} \times \text{currentRotation}$. This prevents error accumulation during a single drag gesture. The snapshot is refreshed on each new mousedown.
- $\text{sensitivity} > 1.0$ amplifies the rotation angle — makes the model feel more responsive to small mouse movements
- In the render loop, the quaternion is converted to a mat4 and multiplied into the view matrix (rotates the camera around the model):

```
mat4.fromQuat(rotMat, arcball.rotation);  
mat4.multiply(view, view, rotMat);    // view = lookAt × rotMat
```

Render Loop

- The view matrix and light position are recomputed every frame:

```
// 1. base lookAt: camera at [0,0,3] looking at origin
mat4.lookAt(view, [0,0,3], [0,0,0], [0,1,0]);

// 2. apply arcball rotation to the view
mat4.fromQuat(rotMat, arcball.rotation);
mat4.multiply(view, view, rotMat);
```

- Light follows the camera — always illuminates the visible face. The camera position in world space is recovered by undoing the rotation:

```
const invRot = transpose(rotMat); // orthogonal → transpose = inverse
const viewPos = transformMat4([0,0,3], invRot);

vec3.add(lightPos, viewPos, [1.5, 1.5, 0]); //light slightly offset from camera
```

Render Loop

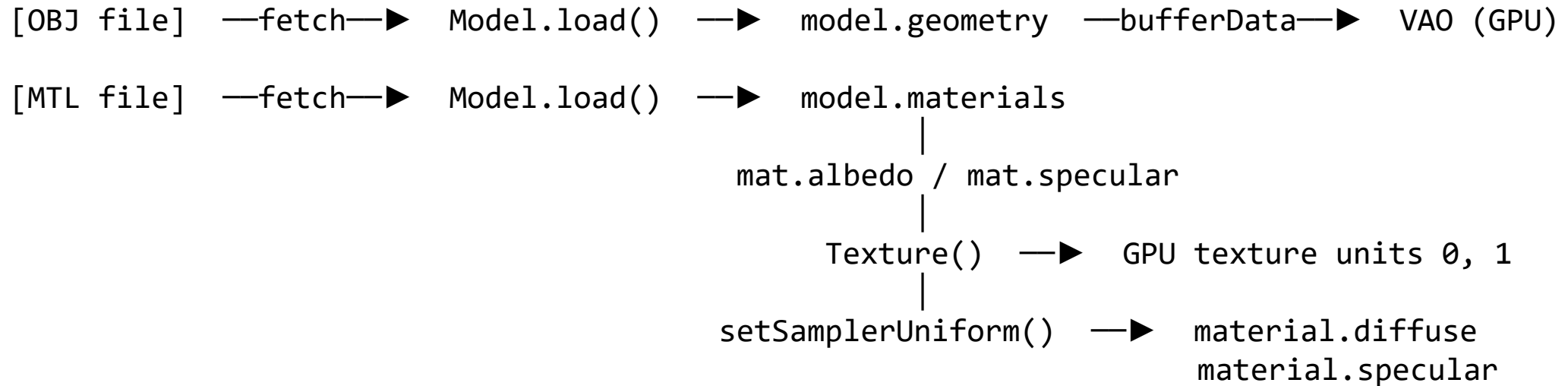
- Draw call: one drawElements for the whole model

```
gl.drawElements(gl.TRIANGLES, mesh.indices.length, gl.UNSIGNED_SHORT, 0);
```

- For multi-material models, replace with a loop over materialRanges:

```
for (const [name, ranges] of Object.entries(mesh.materialRanges)) {  
    bindMaterial(materials[name]);  
    for (const { start, count } of ranges)  
        gl.drawElements(gl.TRIANGLES, count, gl.UNSIGNED_SHORT, start * 2);  
    // *2 = Uint16 byte offset  
}
```


Putting It All Together: Data Flow

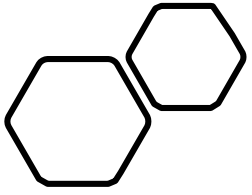


Per frame:

```
mouse drag —► screenToArcball() —► quaternion
quaternion —► mat4 —► view matrix
view matrix —► setMat4('view')
viewPos —► setVec3('viewPos'), setVec3('light.pos')
drawElements(TRIANGLES, indices.length, UNSIGNED_SHORT, 0)
```

Rendered!





Questions???